



The Islamic University of Gaza
Faculty of Engineering
Department of Computer Engineering



ECOM 1101 - Digital Design

Lab #3

Gate-level minimization

Eng. Hashem M. Hejazy

Eng. Rania R. Alsayed

Spring 2025/2026

Introduction

Gate-level minimization aims to find the optimal gate-level implementation of Boolean functions. While manual minimization becomes difficult for circuits with many inputs, computer-based logic synthesis tools handle it efficiently. Understanding the mathematical foundation is essential for designers to perform manual minimization of simple circuits and to use modern design tools effectively. This chapter covers that foundation and introduces a hardware description language.

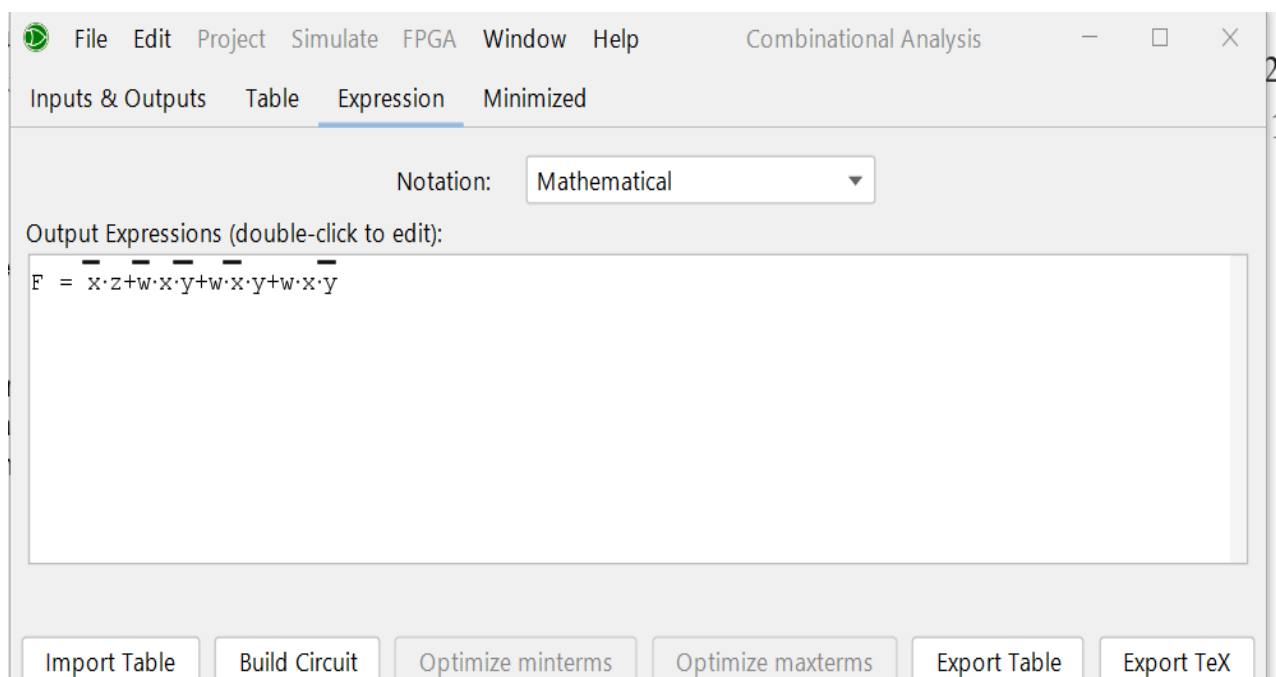
K-map

A K-map is a grid of squares, where each square represents one minterm. Since any Boolean function is a sum of minterms, the function is shown by marking the corresponding squares on the map. The K-map gives a visual picture of all possible simplified forms of the function. By recognizing simple patterns, you can find the simplest expression — meaning the fewest terms and the fewest literals per term. This gives you a circuit with the minimum number of gates and gate inputs. Sometimes you may find two or more equally simple solutions, and any of them is acceptable.

From function to K-map

Ex: Simplify the following Boolean expression:

$$x'z + w'xy' + wx'y + wxy'$$



Output:

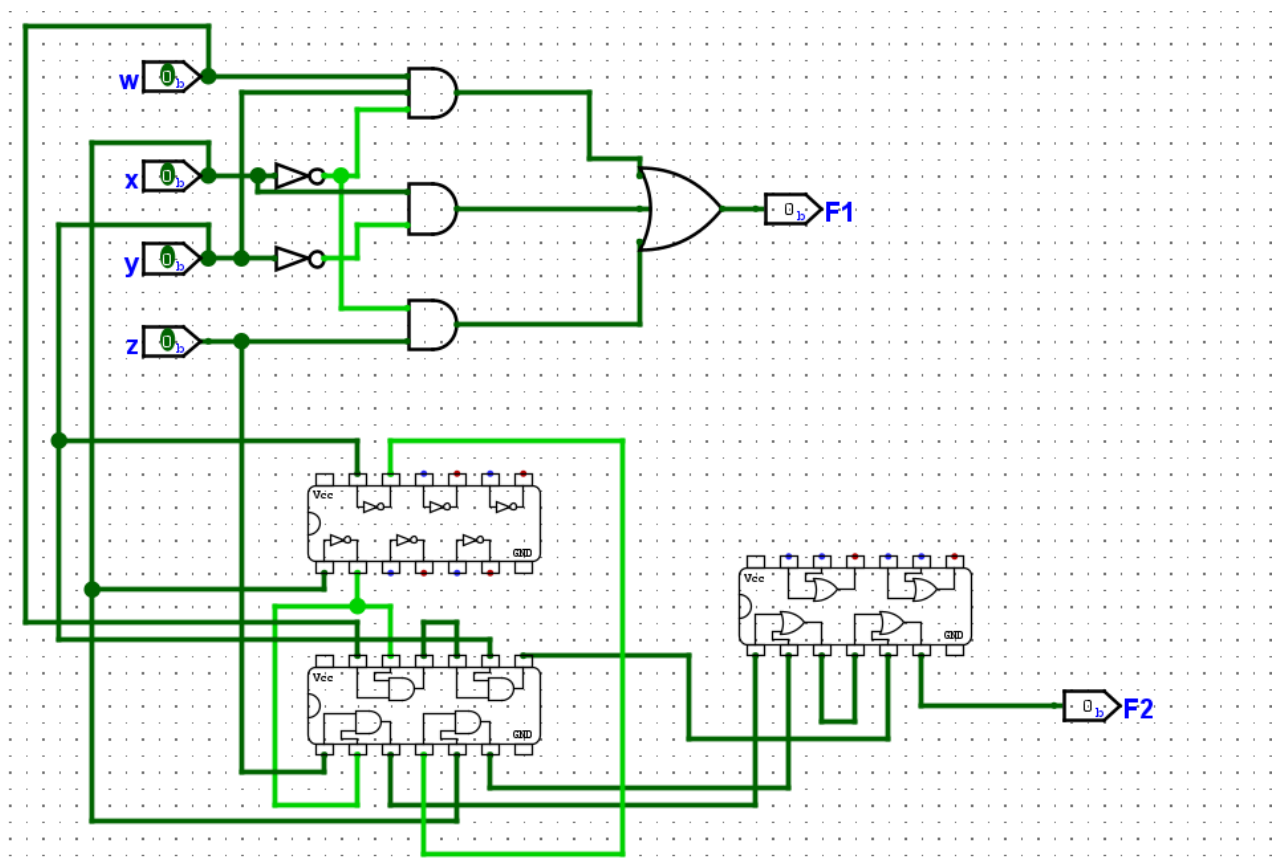
Format:

Style:

Notation:

		y, z			
		00	01	11	10
w, x	00	0	1	1	0
	01	1	1	0	0
	11	1	1	0	0
	10	0	1	1	1

No group selected.

$$\overline{x} \cdot z + x \cdot \overline{y} + w \cdot x \cdot y$$


From Truth table to K-map

Ex 2: $F(w, x, y, z) = \sum(1, 3, 4, 5, 6, 7, 9, 11, 13, 15)$

- 1 0 Collapse Duplicated Rows Show All Rows

16 of 16 rows shown

w	x	y	z	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

Output:

Format:

Style:

Notation:

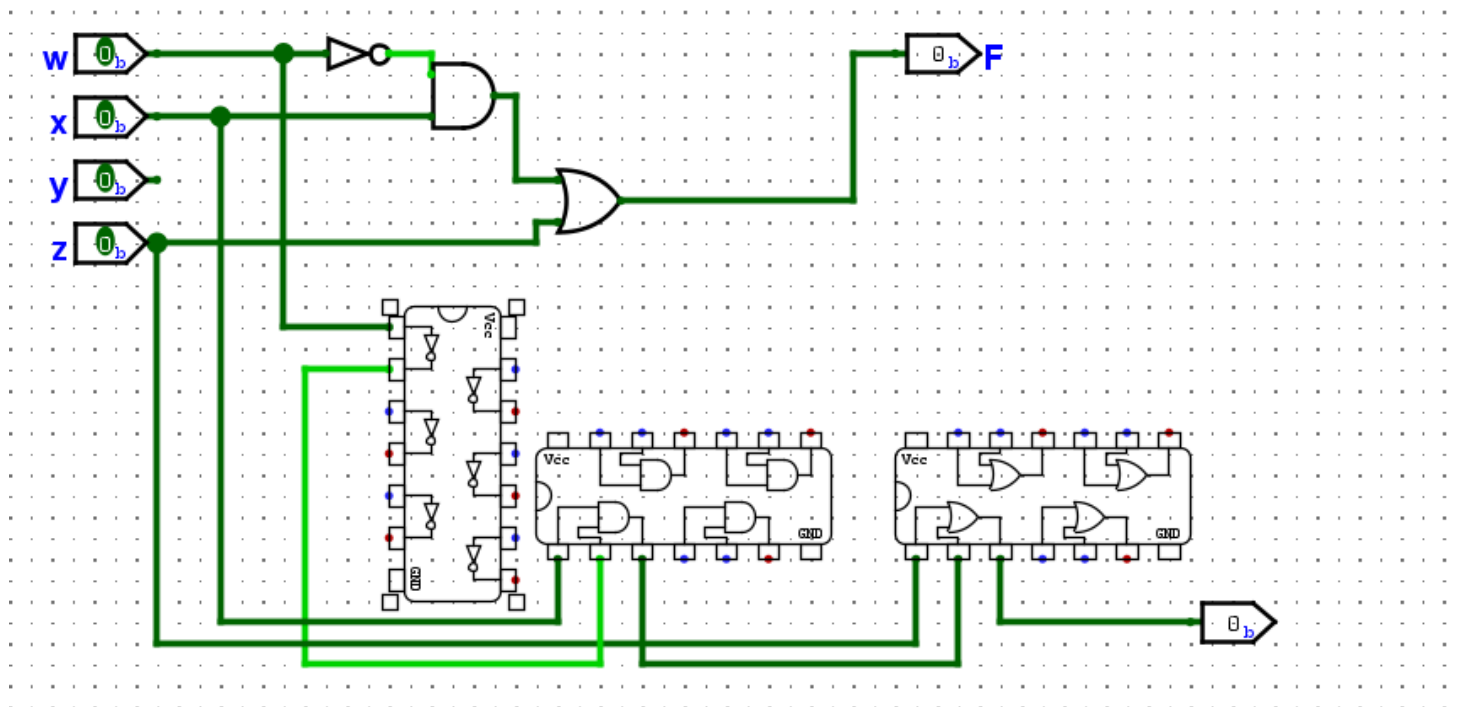
y, z

w, x

	00	01	11	10
00	0	1	1	0
01	1	1	1	1
11	0	1	1	0
10	0	1	1	0

No group selected.

$$\overline{z} + w \cdot x$$



Don't-Care Conditions

In some digital systems, certain input combinations never occur. For these combinations, we don't care what the output is. These are called don't-care conditions and are marked with an X on a K-map. During simplification, you can treat each X as either 0 or 1 — whichever gives a simpler expression. This flexibility often leads to better minimization. However, you are never required to use don't-cares; they are optional tools for simplification. In logisim we represent the don't care as (-).

Ex: $F(A, B, C, D) = \sum (5, 6, 7, 12, 14, 15)$
 $d(A, B, C, D) = \sum (3, 9, 11)$

- 1 0 Collapse Duplicated Rows Show All Rows

16 of 16 rows shown

A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	-
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	-
1	0	1	0	0
1	0	1	1	-
1	1	0	0	1
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

SOP

POS

Output: F

Format: Sum of products

Style: Numbered

Notation: Mathematical

		C, D	00	01	11	10
A, B	00		0	0	-	0
	01		0	1	1	1
	11		1	0	1	1
	10		0	-	-	0

No group selected.

$\overline{A} \cdot B \cdot D + B \cdot C + A \cdot B \cdot \overline{D}$

Output: F

Format: Product of sums

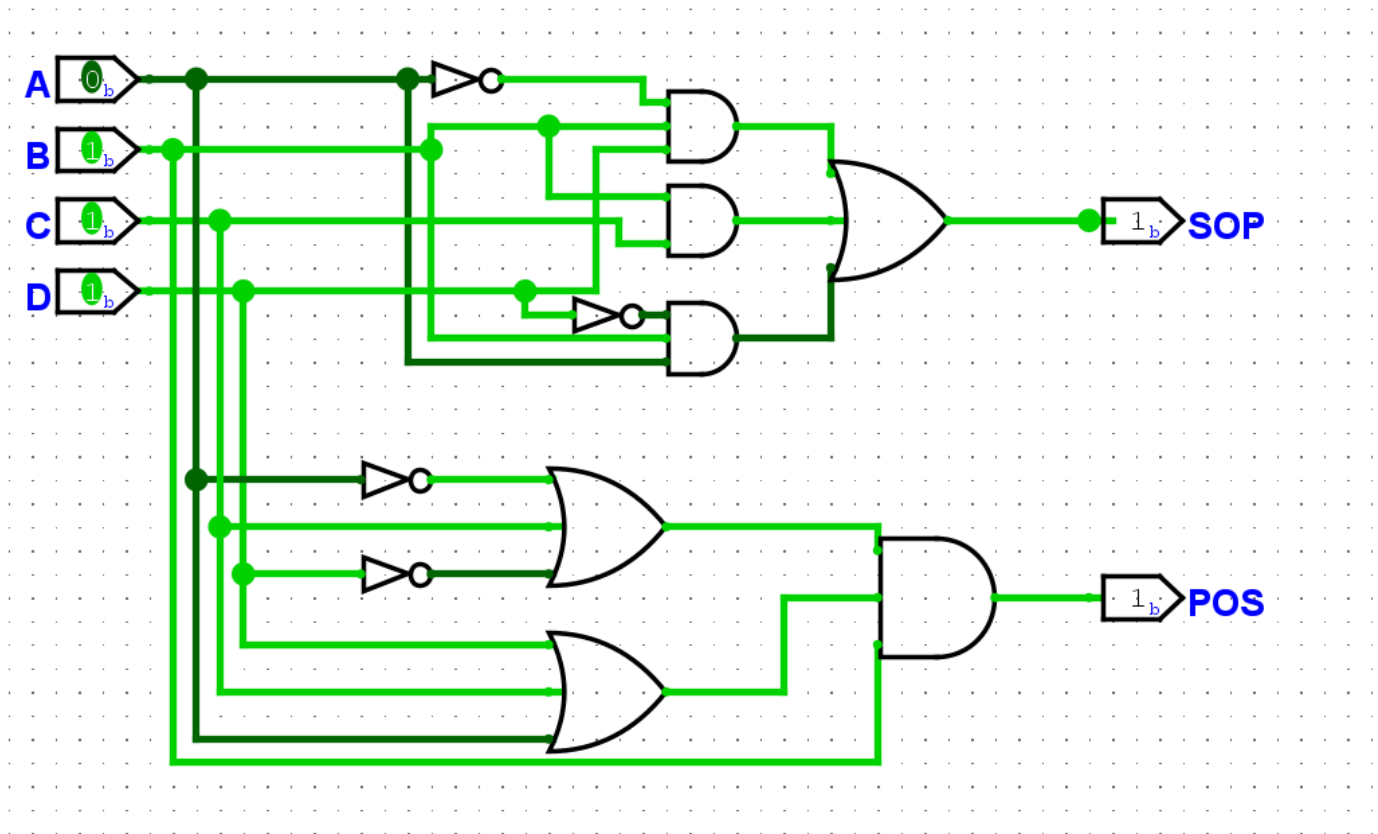
Style: Numbered

Notation: Mathematical

		C, D	00	01	11	10
A, B	00		0	0	-	0
	01		0	1	1	1
	11		1	0	1	1
	10		0	-	-	0

No group selected.

$B \cdot (A+C+D) \cdot (\overline{A}+C+\overline{D})$



Combinational logic circuit design:

A combinational circuit consists of an interconnection of logic gates. Combinational logic gates react to the values of the signals at their inputs and produce the value of the output signal, transforming binary information from the given input data to a required output data. A block diagram of a combinational circuit is shown in Fig below.

Whenever you have been asked to design a circuit that perform some task, use the following procedure steps:

1. From the specifications of the circuit, determine the required number of inputs and outputs and assign a symbol to each.
2. Derive the truth table that defines the required relationship between inputs and outputs.
3. Use K-map to obtain the simplified Boolean functions for each output as a function of the input variables.
4. Draw the logic diagram and verify the correctness of the design.

Ex:

Design a circuit with three inputs (x, y, z) and three outputs (A, B, C). Output A is 1 if the input is greater than 3. Output B is 1 if the input is less than 4. Output C is 1 if the input is equal to 3 or 4.

Step 1: Define Inputs and Outputs

Inputs:

- x,y,z

Outputs:

- A: 1 if input > 3
- B: 1 if input < 4
- C: 1 if input = 3 or 4

Step 2: Truth Table:

x	y	z	A(>3)	B(<4)	C(=3 or 4)
0	0	0	0	1	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	1
1	0	1	1	0	0
1	1	0	1	0	0
1	1	1	1	0	0

Step 3: K-Map Simplification:

Output A

(A = 1 for 4,5,6,7)

Minterms: m(4,5,6,7)

K-map result:

A=x

Output:

Format:

Style:

Notation:

	y, z			
	00	01	11	10
x	0	0	0	0
	1	1	1	1

No group selected.

x

Output B (B = 1 for 0,1,2,3)

Minterms:

m(0,1,2,3)

K-map result:

B=x'

Output:

Format:

Style:

Notation:

	y, z			
	00	01	11	10
x	0	1	1	1
	1	0	0	0

No group selected.

$\bar{x}$

Output C (C = 1 for 3,4)

Minterms: m(3,4)

K-map result:

$$C = x'yz + xy'z'$$

Output:

Format:

Style:

Notation:

	y, z			
	00	01	11	10
x				
0	0	0	1	0
1	1	0	0	0

No group selected.

$\overline{x} \cdot y \cdot z + x \cdot \overline{y} \cdot \overline{z}$

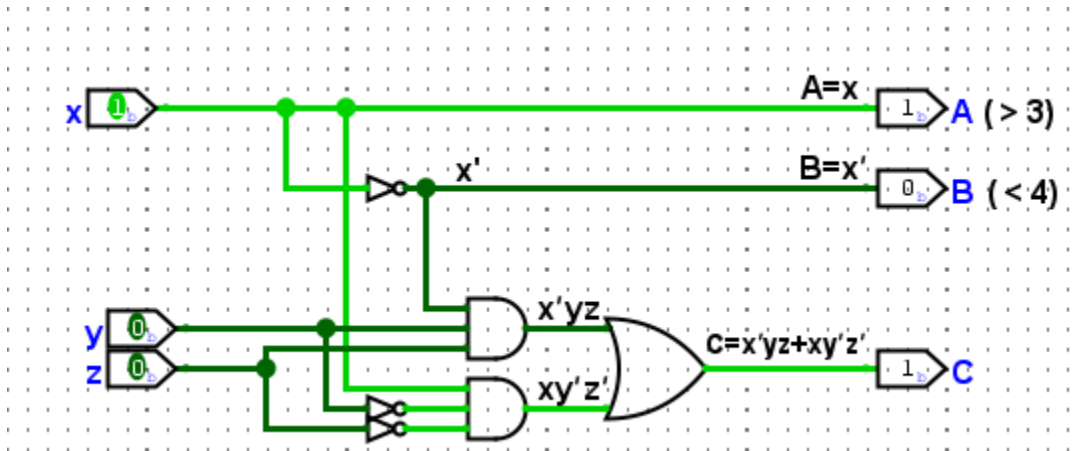
Final Boolean Functions

$$A = x$$

$$B = x'$$

$$C = x'yz + xy'z'$$

Step 4: Draw the logic diagram and verify the correctness of the design.



Homework

Q1: Design a circuit with three inputs (x, y, z) and three outputs (A, B, C). The output is the two's complement of the input only if the input is greater than 3. Otherwise, output equals the input. (Example: 010 (2) → 010; 101 (5) → 011 (3) which is 2's complement of 5? Wait: 5 in 3 bits is 101, 2's complement is 011 which is 3).

Q2: Implement the following Boolean function F , together with the don't-care conditions d.

$$F(A,B,C,D) = \sum(2, 4, 10, 12, 14,)$$

$$d(A, B,C,D) = \sum(0, 1, 5, 8).$$